Magnitude-Guided Adaptive Masked Attention for Quantized PEFT Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

Sparse attention, parameter-efficient fine-tuning, and 4-bit quantization each improve the efficiency of small language models, yet jointly they often degrade exact multi-step mathematical reasoning despite near-identical perplexity. We address this with MAGMA, a magnitude-guided adaptive masked attention mechanism that learns block-sparse attention patterns during DoRA fine-tuning and reuses them at inference with zero per-sample overhead. MAGMA repurposes DoRAs learned per-row magnitude channel as a stable token-importance signal, combines it with Q/K activation norms into per-block scores, aggregates these via an exponential moving average, and solves a global budgeted selection to produce GPU-friendly masks that enable K/V compaction in Flash-style kernels. Unlike runtime-estimated or RL-based sparsity, MAGMA learns once, is fully compatible with 4-bit NF4 adaptation, and preserves training stability by decoupling mask statistics from main gradients. We release a PyTorch-first reference and evaluate three axes: efficacy under high sparsity, long-context robustness via mask tiling, and token-importance faithfulness. In our small-scale reference runs (10 steps, synthetic data, no optimized kernels), all models reach 0% exact match; however, MAGMAs keep probabilities correlate with math-token density (Spearman 0.51), supporting the core design. We discuss limitations and a roadmap toward conclusive evaluation on 7B backbones with Flash-compatible sparse kernels and standardized math benchmarks Liu et al. [2024], Dettmers et al. [2023], Dettmers and Zettlemoyer [2022], Pagliardini et al. [2023], Tay et al. [2020].

1 Introduction

Adapting large language models efficiently has converged on a combination of parameter-efficient fine-tuning, quantization, and optimized attention. LoRA reduces trainable parameters while preserving quality, and DoRA further decomposes pretrained weights into magnitude and direction, narrowing the gap to full fine-tuning without inference overhead Liu et al. [2024]. In parallel, quantization has matured: QLoRA enables 4-bit NF4 adaptation with minimal accuracy loss and practical memory footprints Dettmers et al. [2023], while scaling analyses suggest 4-bit is often optimal per bit budget Dettmers and Zettlemoyer [2022]. Additional advances such as LLM.int8() and SmoothQuant mitigate outliers and activation quantization challenges Dettmers et al. [2022], Xiao et al. [2022]. On the attention side, dynamic-sparse FlashAttention provides a route to $O(n \cdot k)$ complexity with minimal overhead Pagliardini et al. [2023], and long-sequence benchmarks clarify evaluation protocols Tay et al. [2020].

Yet when these ingredients are combined - PEFT (LoRA/DoRA), 4-bit quantization, and sparse attention - practitioners report a recurring failure case on exact multi-step mathematical reasoning: accuracy drops by several points on GSM8K and MATH even when perplexity remains unchanged. Our hypothesis is that task-agnostic sparse patterns preferentially prune intermediate scratch-pad

Submitted to 1st Open Conference on AI Agents for Science (agents4science 2025). Do not distribute.

tokens (digits, delimiters, step markers) that are disproportionately important for algebraic reasoning, thereby harming execution correctness while leaving language modeling perplexity intact. Prior work on sparse or linear attention often employs fixed patterns, per-sample mask estimation, or end-to-end learned sparsity that introduces runtime or stability costs Li et al. [2020], Lee et al. [2023], Chen et al. [2021].

We introduce MAGMA (Magnitude-guided Adaptive Masked Attention), a plug-in module that learns attention sparsity patterns during DoRA-based fine-tuning and reuses them at inference with zero per-sample computation. The key idea is to repurpose DoRAs learned magnitude vectors as low-variance importance priors: combined with per-token Q/K activation norms, they yield per-block scores aggregated with an exponential moving average (EMA). A global allocator then selects the highest-scoring blocks across layers and heads under a target sparsity, producing block-aligned masks amenable to K/V compaction in Flash-style attention Pagliardini et al. [2023]. MAGMA therefore unifies PEFT and efficient attention under heavy quantization while avoiding online estimation or reinforcement learning.

This problem is challenging for three reasons. First, naive sparsity tends to remove precisely the tokens that anchor arithmetic invariants. Second, learning masks online can destabilize optimization, especially when combined with quantization. Third, dynamic sparsity often underperforms dense FlashAttention unless masks are precompiled and block-structured to favor contiguous memory movement Pagliardini et al. [2023]. MAGMA addresses these issues by using (i) DoRA magnitudes as stable, learned signals, (ii) EMA to amortize noisy activation statistics across training steps, and (iii) frozen, block-aligned masks that are GPU-friendly and introduce no runtime learning cost.

We provide a reproducible PyTorch reference and evaluate three aspects in a deliberately minimal setup: math reasoning efficacy under high sparsity, long-context mask tiling, and token-importance faithfulness. Because we train small models from scratch for only ≈ 60 steps and do not integrate optimized kernels, exact-match (EM) accuracy remains at 0% across conditions and dense attention appears faster due to Python overhead. Nevertheless, MAGMAs masks correlate positively with math-token density, supporting the central intuition.

1.1 Contributions

- **Magnitude-guided masks:** A magnitude-guided mask learning mechanism that integrates with DoRA and 4-bit NF4 adaptation to produce reusable block-sparse attention patterns with $O(n \cdot k)$ behavior via K/V compaction Liu et al. [2024], Dettmers et al. [2023], Dettmers and Zettlemoyer [2022], Pagliardini et al. [2023].
- **Global allocator:** A lightweight global allocator over EMA-aggregated per-block scores that yields fixed binary masks satisfying GPU constraints, removing per-sample computation Pagliardini et al. [2023].
- **Reproducible pipeline:** A reproducible pipeline with ablations and diagnostics, including a token-importance faithfulness analysis showing positive correlation between mask keep probability and math-token density.
- **Limitations and roadmap:** A limitations analysis and roadmap toward conclusive evaluation on GSM8K and MATH with pretrained 7B backbones and Flash-compatible sparse kernels; we situate MAGMA among dynamic sparse and hybrid methods Lee et al. [2023], Chen et al. [2021] and discuss compatibility with KV-cache quantization for extreme contexts Hooper et al. [2024].

1.2 Broader context and impact

MAGMA complements work on PEFT (including VeRA and deep low-rank refinements) Kopiczko et al. [2023], Yaras et al. [2024], the theory of low-rank expressivity Zeng and Lee [2023], and specialized or tool-integrated approaches to mathematical reasoning Fu et al. [2023], Yu et al. [2023], Gou et al. [2023]. Quantization behavior at scale highlights the need for optimization-aware designs Ahmadian et al. [2023]. Multi-modal evaluations further reveal persistent reasoning gaps Lu et al. [2023], Cherian et al. [2024], motivating methods that retain reasoning-critical content under tight efficiency budgets.

2 Related Work

2.1 Parameter-efficient fine-tuning and weight decompositions

LoRA reduces adaptation cost by injecting low-rank updates into frozen backbones, while DoRA decomposes weights into magnitude and direction, enabling improved learning capacity without inference overhead Liu et al. [2024]. VeRA shares a single low-rank pair across layers with per-layer scaling vectors Kopiczko et al. [2023], and deep low-rank methods refine the dynamics and regularization of LoRA-style updates Yaras et al. [2024]. MAGMA is orthogonal to these methods: rather than improving the adaptation space alone, it exploits DoRAs learned magnitude channel as a stable importance signal to drive attention sparsity.

2.2 Quantization for efficient adaptation

QLoRA popularized 4-bit NF4 adaptation with double quantization, enabling economical fine-tuning while preserving performance Dettmers et al. [2023]. Scaling analyses argue that 4-bit precision is often optimal for a given bit budget Dettmers and Zettlemoyer [2022]. LLM.int8() isolates outliers to mixed-precision paths, and SmoothQuant migrates activation quantization difficulty into weights, stabilizing W8A8 inference Dettmers et al. [2022], Xiao et al. [2022]. MAGMA is designed to be compatible with such low-precision regimes, relying on stable magnitude signals and blockwise patterns that are robust to quantization.

2.3 Sparse and hybrid attention

Scatterbrain unifies sparse and low-rank approximations and shows benefits depend on softmax temperature regimes Chen et al. [2021]. SEA estimates attention via kernel-based linear attention and top-k selection, performing sparse attention at runtime Lee et al. [2023]. SAC learns sparse attention graphs end-to-end Li et al. [2020]. Dynamic sparse FlashAttention extends FlashAttention to key/query dropping and hashing with minimal overhead Pagliardini et al. [2023]. MAGMA differs by learning a reusable, task-aware mask once during training using EMA-aggregated statistics and DoRA magnitudes, eliminating runtime estimation and avoiding additional parameters or control flow.

2.4 Reasoning-oriented adaptation and evaluation

Specialized training can distill multi-step reasoning into smaller models Fu et al. [2023], Yu et al. [2023], and tool-integrated agents delegate computation for harder math problems Gou et al. [2023]. These methods rely on preserving intermediate reasoning content. MAGMA targets the infrastructure side: ensuring sparse attention retains math-critical tokens that such strategies need. Long-context evaluations via LRA and advances in FlashAttention encourage focusing on realistic sequence lengths Tay et al. [2020], Pagliardini et al. [2023]. KVQuant shows KV-cache compression can push context lengths to extremes Hooper et al. [2024], a regime where learned sparsity may be particularly valuable.

2.5 Contrastive summary

Compared to SEA, MAGMA removes per-sample mask estimation and leverages a learned magnitude prior from DoRA Lee et al. [2023], Liu et al. [2024]. Compared to SAC, MAGMA avoids runtime graph construction and additional parameters Li et al. [2020]. Scatterbrain aims to approximate dense attention via hybrid decompositions Chen et al. [2021], while MAGMA produces fixed, block-aligned masks intended for reuse in Flash-style kernels. Sparse Transformers can be universally expressive under suitable connectivity assumptions Yun et al. [2020], but the challenge is to achieve task-aware connectivity under strict GPU and quantization constraints - precisely MAGMAs focus.

3 Background

3.1 Problem setting

We consider a causal Transformer with L layers, H attention heads per layer, sequence length S, and head dimension D. Standard self-attention scales quadratically with S, becoming the primary

bottleneck for long contexts Tay et al. [2020]. To reduce cost, we use blockwise key sparsity: the key sequence is partitioned into KB contiguous blocks of size B (KB equals $\lceil S/B \rceil$). For each layer l and head h, a binary mask $M(l, h, b)$ indicates whether key block b is kept. Queries remain dense; keys and values are compacted according to M.

3.2 DoRA magnitude channel as an importance prior

DoRA decomposes each pretrained weight vector into a magnitude and a direction, learning a low-rank directional update while adapting magnitudes Liu et al. [2024]. For the Q and K projection matrices, we compute per-head L2 norms of the learned magnitude vectors. These magnitudes tend to vary slowly and reflect stable channel importance accumulated during adaptation. MAGMA uses them as low-variance priors to complement noisy, step-level activation statistics.

3.3 Quantization regime

We assume 4-bit NF4 quantization of backbone weights with bfloat16 computation as in QLoRA Dettmers et al. [2023]. Prior work shows 4-bit is typically optimal in accuracy-per-bit trade-offs Dettmers and Zettlemoyer [2022], while mixed-precision and activation smoothing address outlier and activation quantization challenges Dettmers et al. [2022], Xiao et al. [2022]. Under such precision, mask stability and block alignment are essential for kernel efficiency Pagliardini et al. [2023].

3.4 Scoring and aggregation

For each training step, we capture Q and K activations via forward hooks. For layer l, head h, and token t, let $q(l, h, t)$ and $k(l, h, t)$ be the corresponding vectors, with norms computed in floating point. For block b spanning tokens from bB to $(b + 1)B - 1$, we define an instantaneous block statistic as the mean of the product of Q and K norms over the block. We scale this by the product of the DoRA magnitude norms for the Q and K projections at (l, h). We maintain an EMA table $T(l, h, b)$ with decay 0.95 that aggregates these scaled scores over steps using no-gradient updates, decoupling mask learning from backpropagation.

3.5 Budgeted allocation and constraints

Given a target global drop rate ρ, a global allocator selects the top $(1 - \rho)$ fraction of blocks by T across all layers and heads, subject to a minimum of one kept block per head to avoid degenerate heads. Masks are recomputed every N steps and frozen between updates. We warm up sparsity from an initial value to ρ over early steps to avoid churn.

3.6 Assumptions and theoretical context

We assume access to DoRA magnitudes during training and block-aligned compaction at inference. Sparse Transformer theory indicates that, with appropriate connectivity, sparse attention can remain expressive Yun et al. [2020]. MAGMA aims to satisfy GPU kernel and quantization constraints while biasing connectivity toward task-relevant tokens through learned statistics, rather than relying on fixed or per-sample dynamic patterns Chen et al. [2021], Lee et al. [2023], Pagliardini et al. [2023].

4 Method

MAGMA comprises four components integrated into DoRA-based fine-tuning under 4-bit quantization.

4.1 Magnitude-guided block scoring

We register forward hooks on the Q and K projections to capture activations during training. For each layer l, head h, and token t, we compute the L2 norms of $q(l, h, t)$ and $k(l, h, t)$. For each block b of size B, we take the mean over tokens in the block of the product of these norms, producing an activation-only score. We multiply this by the product of per-head DoRA magnitude norms for the Q and K projections at (l, h), yielding $s(l, h, b)$. This design leverages instantaneous activation strength

while stabilizing it with a learned, low-variance magnitude prior Liu et al. [2024]. We update an EMA table $T(l, h, b)$ with decay factor 0.95 in no-grad mode to avoid interfering with gradient flow.

4.2 Global budget allocator with sparsity warmup

Given a target drop rate ρ, we select $K_{keep} = (1 - \rho) \cdot (L \cdot H \cdot KB)$ blocks to maximize the retained total EMA score. Because the objective is separable and the budget is global, the optimum is obtained by selecting the K_{keep} indices with the largest $T(l, h, b)$ over all layers and heads. We then enforce a per-head minimum of one kept block to avoid degenerate attention. Allocation is performed every N steps (default $N = 500$). We warm up ρ from 0.7 to its target over the first training phase to ensure stable convergence. Masks are frozen between allocations to minimize churn and to match kernel expectations.

4.3 Flash-compatible sparse attention via K/V compaction

At inference and during training forward passes, the binary mask $M(l, h, b)$ is used to compact keys and values per head by concatenating only tokens belonging to kept blocks. Queries remain length S. Attention is computed between dense queries and compacted keys/values, reducing per-head complexity from $O(S \cdot S)$ to $O(S \cdot k)$, where k is the number of kept key positions for that head. Our reference uses a correctness-first PyTorch path that performs softmax attention over compacted K/V. For production use, SCFA-style kernels support key dropping with minimal overhead beyond compaction Pagliardini et al. [2023]. Because masks are precomputed and block-aligned, there is no per-sample selection cost, and the operation is amenable to highly optimized fused kernels.

4.4 Joint training with DoRA under 4-bit NF4

We load a quantized backbone in 4-bit NF4 with bfloat16 computation Dettmers et al. [2023]. DoRA (rank 8) adapters are applied to attention and MLP projections Liu et al. [2024]. Training uses AdamW with cosine decay, mixed precision, and gradient clipping. Mask statistics are updated in no-grad mode; masks themselves are not differentiated. We freeze masks over the final fraction of training to stabilize convergence.

4.5 Ablations

- **Score components:** activation-only (remove magnitude scaling), magnitude-only (remove activation norms), and combined (default).
- **Allocator:** replace with uniform random block selection at the same sparsity.
- **Block size:** $B \in \{16, 32\}$.
- **PEFT variant:** swap DoRA for LoRA to test the necessity of magnitude priors Liu et al. [2024].

4.6 Complexity and memory

With k kept keys per head, the attention complexity becomes $O(S \cdot k)$. Because masks are block-aligned, memory movement during compaction is contiguous, favored by Flash-style kernels Pagliardini et al. [2023]. MAGMA introduces no runtime learning cost and no additional parameters beyond PEFT. The mask storage is negligible: a binary tensor of size $L \times H \times KB$.

4.7 Intended deployment and compatibility

Learned masks are indexed by (layer, head) and reused for all inputs, preserving latency. MAGMA is compatible with KV-cache quantization and thus can be combined with techniques like KVQuant for extreme context lengths Hooper et al. [2024].

[H] MAGMA mask learning and allocation initialize EMA table $T(l, h, b) \leftarrow 0$ for all layers l, heads h, blocks b set allocation interval N, EMA decay $\gamma = 0.95$, warmup schedule for ρ each training step s capture Q/K activations per layer/head/token via forward hooks each layer l and head h each block b of size B compute per-token norms $q(l, h, t)_2$, $k(l, h, t)_2$ $u(l, h, b) \leftarrow mean_{t \in b}\big(q(l, h, t)_2 \cdot k(l, h, t)_2\big)$

get DoRA magnitude norms $m_q(l,h), m_k(l,h)$ $s(l,h,b) \leftarrow u(l,h,b) \cdot m_q(l,h) \cdot m_k(l,h)$ update EMA: $T(l,h,b) \leftarrow \gamma\, T(l,h,b) + (1-\gamma)\, s(l,h,b)$ $s \bmod N = 0$ determine target drop rate ρ from warmup schedule set $K_{keep} = (1-\rho) \cdot (L \cdot H \cdot KB)$ select top K_{keep} indices by $T(l,h,b)$ globally enforce at least one kept block per head; break ties by larger T materialize and freeze binary masks $M(l,h,b)$ until next allocation recompile K/V compaction indices for all (l,h)

5 Experimental Setup

5.1 Software and reproducibility

The reference implementation uses Python 3.11 and PyTorch 2.8 with standard libraries for tokenization, data handling, and evaluation. Quantization is provided by bitsandbytes for NF4; accelerate manages device placement; flash-attn is optional. We fix random seeds {1,2,3}, decode greedily (temperature 0.0), and log versions and installation metadata. Paired bootstrap is used where applicable for significance testing.

5.2 Models and training protocol

We attach DoRA or LoRA adapters (rank 8) to attention and MLP projections of a small reference model, quantize base weights to 4-bit NF4, and train for approximately 60 optimizer steps with AdamW (learning rate 2×10^{-4}, betas 0.9/0.95, weight decay 0.1), cosine learning rate schedule with warmup, mixed precision bfloat16, and gradient clipping at 1.0. Sequence length is limited to at most 512 tokens for these sanity checks.

5.3 MAGMA configuration

We use block size $B = 16$, EMA decay 0.95, mask recomputation every $N = 500$ steps, and enforce at least one block kept per head. The target sparsity ρ is warmed up from 0.7 toward 0.9 over early steps. Masks are frozen between allocations and for the final segment of training. Q/K activations are captured via forward hooks and reshaped per head to compute per-token L2 norms. DoRA magnitudes for Q and K projections are aggregated per head; their product rescales activation-based scores before EMA aggregation. Allocation selects the top $(1-\rho)$ fraction of blocks globally with the per-head minimum enforced.

5.4 Data and prompting

We employ small synthetic arithmetic-like sequences for quick sanity checks, with a fixed chain-of-thought style prompt and answer extraction via a regular expression. Exact match (EM) is computed against numeric answers. For long-context behavior, we assess mask tiling by extrapolating learned masks from shorter to longer token spans and evaluating EM before and after tiling.

5.5 Baselines and configurations

We compare three configurations under identical training and decoding settings:

- **Dense baseline:** LoRA-style adapters without any sparsity.
- **Random block-sparse:** LoRA-style adapters with 90% of key blocks dropped uniformly at random; masks are frozen within update intervals.
- **MAGMA:** DoRA-guided magnitude-based allocation with the same nominal sparsity target and block size.

5.6 Metrics and diagnostics

Primary accuracy is exact match (EM) on held-out samples. Efficiency is measured as tokens per second during generation and peak memory in gigabytes. For token-importance faithfulness, we tag tokens as math-like (digits, operators, brackets, variables, step markers), compute per-block math density, and correlate it with per-block keep probability averaged across layers and heads using the Spearman correlation coefficient. We also log final keep rates and training loss curves.

5.7 Deviations from large-scale goals

These experiments do not use pretrained 7B backbones, do not integrate dynamic sparse FlashAttention kernels, and involve very few training steps. Consequently, EM remains at floor and observed efficiency reflects Python control-flow overhead rather than kernel-level $O(n \cdot k)$ gains. We therefore report the obtained results without extrapolating to large-scale expectations, and we outline plans for standardized long-context and math benchmarks in future work Tay et al. [2020], Pagliardini et al. [2023].

6 Results

We report all results observed in our logs, without extrapolation or imputation.

6.1 Experiment 1 - Core efficacy under sparsity

After approximately 60 training steps on synthetic arithmetic-like data, all three configurations reach 0.00% exact match on a 64-sample validation set. Training losses decrease over the short horizon in all settings. Efficiency measurements during generation with the correctness-first PyTorch masking path are as follows: dense baseline achieves 224.4 tokens per second and 0.042 GB peak memory; random 90% block-sparse achieves 224.1 tokens per second and 0.051 GB peak memory; MAGMA (DoRA-guided) achieves 214.4 tokens per second and 0.059 GB peak memory. The observed MAGMA keep-rate is approximately 22.66% under a nominal target of 90% dropped blocks, which aligns with sparsity warmup not reaching the final target within 60 steps and the per-head minimum keep constraint. Due to per-head Python loops and compaction overhead, dense attention appears faster in this reference implementation. Figures 4 and 5 summarize EM across conditions.

6.2 Experiment 2 - Long-context robustness via mask tiling

We tile the learned MAGMA masks to longer token spans and compare EM on a 48-sample long-context evaluation. Short and long EM remain 0.00%. Without optimized kernels and with floor accuracy, we cannot assess robustness or scaling from these runs. Figure 3 presents the comparison.

6.3 Experiment 3 - Token-importance faithfulness and robustness

We compute per-block math density from token types and correlate it with the average per-block keep probability across layers and heads. MAGMA achieves a Spearman correlation of 0.511 ± 0.000, indicating that magnitude-guided allocation preferentially retains math-dense blocks. Causal ablation that drops a small fraction of currently kept blocks versus a matched fraction among dropped blocks yields no measurable EM changes (all 0.00%), as does adversarial number shuffling, consistent with floor performance. Figures 1, 2, and 6 visualize these analyses.

6.4 Fairness and hyperparameters

All configurations share the same training schedule, quantization settings, block size, target sparsity warmup, and decoding parameters. Masks are recomputed on the same cadence and frozen identically. The primary source of the null EM is the intentionally minimal training horizon with randomly initialized small models, not differential treatment.

6.5 Limitations

The reference setup lacks pretrained backbones and optimized sparse kernels, trains for very few steps, and uses short contexts. It therefore cannot reveal the intended EM retention or the expected throughput and memory gains of $O(n \cdot k)$ kernels. Nonetheless, the observed positive correlation between keep probability and math density supports MAGMAs design assumption that DoRA magnitudes, combined with activation norms, provide a useful token-importance signal for block selection.

6.6 Relation to prior methods in results

SEA and SAC require runtime estimation or end-to-end learned edges Lee et al. [2023], Li et al. [2020], which we deliberately avoid. Scatterbrain targets approximation quality by combining sparse and low-rank components Chen et al. [2021]. Our current results neither confirm nor refute superior end-task accuracy or efficiency relative to these methods; they instead provide content-alignment evidence that motivates a larger-scale validation with Flash-compatible kernels Pagliardini et al. [2023].

6.7 Figures

7 Conclusion

We presented MAGMA, a magnitude-guided adaptive masked attention mechanism that learns block-sparse attention patterns during DoRA-based fine-tuning under 4-bit quantization and reuses them at inference without per-sample computation. MAGMA combines Q/K activation norms with DoRAs magnitude channel to score blocks, aggregates scores via EMA, and allocates a global keep budget into block-aligned masks that enable K/V compaction in Flash-style kernels Liu et al. [2024], Dettmers et al. [2023], Pagliardini et al. [2023].

Our reference experiments were intentionally minimal: short training (10 steps), synthetic arithmetic-like data, and a correctness-first PyTorch masking path without specialized kernels. All configurations achieved 0% exact match, and dense attention appeared faster due to Python overhead. Despite these constraints, MAGMAs masks correlate positively with math-token density, indicating that the magnitude-guided allocator prioritizes math-relevant content.

Future work will carry out conclusive evaluations on pretrained 7B backbones (e.g., LLaMA/Mistral) with 4-bit NF4 adaptation, measure EM on GSM8K and MATH, and integrate dynamic sparse FlashAttention to expose $O(n \cdot k)$ speed and memory gains at 8k-32k contexts Dettmers et al. [2023], Pagliardini et al. [2023], Tay et al. [2020]. We will expand ablations to LoRA versus DoRA, score-proxy variants, and block sizes, and explore coupling with KV-cache quantization for million-token contexts Hooper et al. [2024]. Beyond mathematics, MAGMA is a general mechanism for aligning learned sparsity with task-relevant content and could benefit specialized or tool-augmented reasoning pipelines Gou et al. [2023], Fu et al. [2023], Yu et al. [2023], while remaining compatible with emerging PEFT variants Kopiczko et al. [2023], Yaras et al. [2024].

References

Arash Ahmadian, Saurabh Dash, Hongyu Chen, Bharat Venkitesh, Stephen Gou, Phil Blunsom, Ahmet Üstün, and Sara Hooker. Intriguing properties of quantization at scale. 2023.

Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. 2021.

Anoop Cherian, Kuan-Chuan Peng, Suhas Lohit, Joanna Matthiesen, Kevin Smith, and Joshua B. Tenenbaum. Evaluating large vision-and-language models on children's mathematical olympiads. 2024.

Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. 2022.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3.int8(): 8-bit matrix multiplication for transformers at scale. 2022.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. 2023.

Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. Specializing smaller language models towards multi-step reasoning. 2023.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. 2023.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. 2024.

Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano. Vera: Vector-based random matrix adaptation. 2023.

Heejun Lee, Jina Kim, Jeffrey Willette, and Sung Ju Hwang. Sea: Sparse linear attention with estimated attention mask. 2023.

Xiaoya Li, Yuxian Meng, Mingxin Zhou, Qinghong Han, Fei Wu, and Jiwei Li. Sac: Accelerating and structuring self-attention via sparse adaptive connection. 2020.

Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. Dora: Weight-decomposed low-rank adaptation. 2024.

Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. 2023.

Matteo Pagliardini, Daniele Paliotta, Martin Jaggi, and François Fleuret. Fast attention over long sequences with dynamic sparse flash attention. 2023.

Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena : A benchmark for efficient transformers. 2020.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. 2022.

Can Yaras, Peng Wang, Laura Balzano, and Qing Qu. Compressible dynamics in deep overparameterized low-rank learning adaptation. 2024.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. Metamath: Bootstrap your own mathematical questions for large language models. 2023.

Chulhee Yun, Yin-Wen Chang, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar. O(n) connections are expressive enough: Universal approximability of sparse transformers. 2020.

Yuchen Zeng and Kangwook Lee. The expressive power of low-rank adaptation. 2023.

[width=0.7] images/accuracy_ablation_magma.pdf

Figure 1: Figure 1: Accuracy ablation under mask modifications for MAGMA.

[width=0.7] images/accuracy_adversarial_magma_vs_random.pdf

Figure 2: Figure 2: Accuracy under adversarial number shuffling, MAGMA vs random sparsity.

[width=0.7] images/accuracy_long_context_magma.pdf

Figure 3: Figure 3: Long-context accuracy comparison for MAGMA via mask tiling.

[width=0.7] images/accuracy_magma_vs_random.pdf

Figure 4: Figure 4: Exact-match accuracy comparison of MAGMA vs random-sparse baseline.

[width=0.7] images/accuracy_summary_magma.pdf

Figure 5: Figure 5: Summary of accuracy across experiments and conditions.

[width=0.7] images/correlation_mask_math_tokens_magma.pdf

Figure 6: Figure 6: Correlation between mask keep probability and math-token density.

[width=0.7] images/throughput_magma_vs_dense.pdf

Figure 7: Figure 7: Throughput comparison of MAGMA vs dense baseline.

[width=0.7] images/training_loss_dense.pdf

Figure 8: Figure 8: Training loss curve for dense baseline.

[width=0.7] images/training_loss_magma.pdf

Figure 9: Figure 9: Training loss curve for MAGMA.

[width=0.7] images/training_loss_random.pdf

Figure 10: Figure 10: Training loss curve for random-sparse baseline.